Table des matières

Exercices corrigés 11 — Transactions et concurrence 1
Questions courtes 1
Questions pratiques 2
Diagnostics 3
Questions à développer 4
Mises en situation 5
Corrigé 6

Exercices corrigés 11 — Transactions et concurrence

Comment travailler ces exercices. Répondez à chaque énoncé seul, puis vérifiez. Les questions courtes vérifient que le vocabulaire est en place ; les pratiques vous font écrire du SQL, souvent à deux sessions ; les diagnostics vous montrent du code qui cloche ; les mises en situation vous demandent de choisir une approche et de la défendre. Comptez trois heures et quart pour y répondre, le corrigé non compris.

Les repères de terrain. Ce module n'a qu'un laboratoire, sur PostgreSQL. Un énoncé marqué `[OLTP]` se traite donc dans PostgreSQL, sur le schéma `labo11` que vous avez construit. Les rares énoncés marqués `[OLAP]` se traitent dans DuckDB, sur l'entrepôt. Sans marque, la question ne demande pas de moteur.

Le corrigé suit les énoncés, en fin de document. Répondez à l'énoncé en entier avant d'y passer : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. Définissez une transaction en une phrase, sans employer le mot ACID.
2. Un chargement vide une table puis la remplit, sans transaction. Décrivez précisément ce que voit un rapport lancé entre les deux commandes, et pourquoi personne ne s'en apercevra.
3. Que garantit la lettre C d'ACID, et pourquoi ne vous protège-t-elle de rien si vous n'avez rien déclaré ?
4. Laquelle des quatre lettres d'ACID se règle, et pourquoi les trois autres ne se règlent-elles pas ?
5. À quoi sert un `SAVEPOINT` ? Donnez la situation où il devient indispensable sur PostgreSQL.

© 2026 iAlexMG. Tous droits réservés.

6. On lit souvent que le DDL est validé d'office. Sur quel moteur de cette formation est-ce faux, et qu'est-ce que cela permet de faire ?
7. Nommez les cinq anomalies de concurrence, et dites laquelle des cinq ne se produit jamais sur PostgreSQL.
8. En quoi la mise à jour perdue et l'anomalie d'écriture diffèrent-elles des trois autres anomalies ?
9. Racontez l'anomalie d'écriture en trois phrases, avec l'exemple des médecins de garde.
10. Quel est le niveau d'isolation par défaut de PostgreSQL, et qu'autorise-t-il encore ?
11. En quoi la lecture répétable de PostgreSQL est-elle plus stricte que ce que la norme exige ?
12. Le niveau sérialisable protège d'une façon particulière. Laquelle, et quelle obligation cela crée pour l'application ?
13. Différence entre un verrou partagé et un verrou exclusif, entre un verrou de ligne et un verrou de table.
14. Un blocage et un interblocage. Lequel est une erreur, lequel se résout tout seul, et comment ?
15. Donnez la règle en une phrase qui évite la plupart des interblocages.
16. Que fait PostgreSQL quand vous lancez un `UPDATE` sur une ligne ? Décrivez le mécanisme, pas le résultat.
17. Citez deux conséquences visibles du MVCC, une agréable et une désagréable.
18. Pourquoi un parcours d'index seul affiche-t-il parfois des accès à la table, et qu'est-ce qui fait baisser ce compteur ?
1. Pourquoi la concurrence se pose-t-elle autrement sur un entrepôt ? Le voyage dans le temps remplace-t-il la transaction ?
2. Définissez un chargement idempotent, citez les trois motifs qui le garantissent, et donnez la condition que le `MERGE` impose à la source.

Questions pratiques

P1. `[OLTP]` À deux sessions, produisez une lecture non répétable, puis montrez que le niveau de lecture répétable la fait disparaître. Écrivez les blocs dans l'ordre, en indiquant quelle session exécute quoi.

P2. `[OLTP]` Écrivez la transaction qui débite un compte de 100 $ et crédite l'autre, en refusant le débit si le solde devient négatif. Vous devez traiter le cas d'échec sans laisser la base à mi-chemin.

P3. `[OLTP]` Produisez une mise à jour perdue à deux sessions, puis corrigez-la de deux façons différentes. Dites dans quel cas chaque correction est la bonne.

P4. `[OLTP]` Écrivez la requête qui répond à « quelle session bloque quelle autre en ce moment », en donnant pour chacune son numéro de processus et le début de sa commande.

© 2026 iAlexMG. Tous droits réservés.

P5. [OLTP] Provoquez un interblocage à deux sessions, puis réécrivez le même scénario de façon qu'il ne produise qu'un blocage. Expliquez ce que vous avez changé.

P6. [OLTP] Rendez ce chargement rejouable et sûr pour les lecteurs. Écrivez la version corrigée et justifiez chaque ajout :

```
DELETE FROM evenement WHERE jour = '2024-03-01';
INSERT INTO evenement (jour, source, montant) SELECT '2024-03-01', source, montant FROM arrivee;
```

P7. [OLTP] Écrivez le `MERGE` qui met à jour la table `cible` depuis `source` sur la clé `cle`, plus la requête de contrôle qui doit s'exécuter avant lui. Dites ce qui se passe si vous omettez le contrôle. (avancé)

P8. Une commande a supprimé des lignes par erreur il y a dix minutes. Dites comment vous constatez l'état d'avant et comment vous restaurez, sur chacun des deux terrains de la formation, puis sur un grand entrepôt. Les trois réponses sont différentes, et une seule ne demande rien à préparer d'avance : dites laquelle et pourquoi les deux autres obligent à y avoir pensé avant. (avancé)

Diagnostics

E1. Un script de chargement quotidien :

```
TRUNCATE TABLE rapport_ventes;
INSERT INTO rapport_ventes SELECT ... FROM ...;
```

Il tourne à 2 h du matin depuis trois ans. Un utilisateur signale qu'une fois par mois environ, le tableau de bord affiche zéro pendant quelques minutes. Expliquez, et corrigez.

E2. Une application bancaire :

```
-- Lu par l'application
SELECT solde FROM compte WHERE id = 1;
-- Calculé côté application : nouveau_solde = solde - montant
UPDATE compte SET solde = 350 WHERE id = 1;
```

Nommez l'anomalie possible, décrivez le scénario exact qui la produit, et donnez deux corrections.

E3. Un développeur veut protéger un rapport et écrit :

```
BEGIN ISOLATION LEVEL SERIALIZABLE;
SELECT ... ;   -- requête 1
SELECT ... ;   -- requête 2
COMMIT;
```

Le rapport échoue maintenant une fois sur vingt avec une erreur de sérialisation. Expliquez pourquoi, et proposez la correction la plus économique.

© 2026 iAlexMG. Tous droits réservés.

E4. Une migration de schéma lancée à 10 h du matin :

```
BEGIN;
ALTER TABLE commande ADD COLUMN canal text;
UPDATE commande SET canal = 'web' WHERE ...;   -- 40 millions de lignes, 25 minutes
COMMIT;
```

L'application est figée pendant vingt-cinq minutes, sans aucune erreur dans les journaux. Expliquez, et dites ce que vous auriez fait autrement.

E5. Une table d'audit de 900 millions de lignes est mise à jour en continu. Ses parcours ont doublé de durée en six mois, alors que le nombre de lignes n'a augmenté que de dix pour cent, et qu'aucun index n'a changé. Donnez la cause la plus probable, la requête qui la confirme, et les deux traitements possibles avec leur coût respectif. (avancé)

E6. Un développeur écrit ce bloc, et il pensait pouvoir tout annuler :

```
BEGIN;
CREATE OR REPLACE TABLE cible AS SELECT * FROM source;
DELETE FROM journal WHERE traite = false;
ROLLBACK;
```

Dites ce qui est réellement annulé sur chacun des deux terrains de la formation, ce qui le serait sur un moteur qui valide le DDL d'office, et pourquoi. Terminez par la règle d'écriture qui rend ce bloc sûr partout, sans rien supposer du moteur.

Questions à développer

A1. La théorie affirme que l'isolation est la seule des quatre lettres d'ACID qui se règle. Expliquez en un paragraphe ce que ce réglage échange contre quoi, puis décrivez comment vous choisiriez le niveau pour trois usages distincts : une application de caisse, un rapport analytique multi-requêtes, et un chargement nocturne.

A2. Le module affirme qu'un chargement doit être idempotent. Défendez cette exigence en un paragraphe, en donnant au moins trois raisons concrètes pour lesquelles un chargement finit toujours par être relancé. Comparez ensuite les trois motifs d'idempotence et dites lequel vous choisiriez pour une table de faits de 200 millions de lignes alimentée par jour.

A3. Comparez la protection offerte par une transaction et celle offerte par le voyage dans le temps d'un entrepôt. Votre réponse doit dire ce que chacune empêche, ce que chacune répare, et pourquoi l'une ne remplace pas l'autre. Concluez sur ce qu'un ingénieur analytique doit mettre en place quand il ne dispose que de la seconde.

A4. Le niveau sérialisable garantit qu'aucune anomalie ne passe, et pourtant il est rarement employé. Exposez les raisons de cette rareté, dites dans quelles conditions vous le recommanderiez malgré tout, et décrivez ce qu'il faut avoir écrit dans l'application avant de l'activer. (avancé)

© 2026 iAlexMG. Tous droits réservés.

Mises en situation

M1. Billetterie Val-Jalbert, la surréservation du samedi. Val-Jalbert vend des billets pour des spectacles à jauge limitée. Le code de réservation compte les billets déjà vendus, vérifie qu'il reste de la place, puis insère la réservation. Le samedi soir, à l'ouverture des ventes d'un spectacle populaire, la salle se retrouve régulièrement avec trois à huit billets de trop. Le reste de la semaine, tout va bien.

Nommez l'anomalie en cause et expliquez précisément le scénario qui la produit, en décrivant ce que fait chaque session. Proposez ensuite trois corrections de natures différentes, dites ce que chacune coûte en débit un samedi soir, et recommandez-en une. Précisez enfin ce que vous mesureriez pour vérifier que le problème a disparu, sachant qu'il ne se manifestait déjà qu'une fois par semaine.

M2. Entrepôt logistique Nicolet, l'application qui se fige. Chaque matin entre 8 h 10 et 8 h 40, l'application de préparation de commandes se fige pour la moitié des utilisateurs. Aucune erreur n'apparaît dans les journaux applicatifs, aucun message dans ceux de la base, et le processeur du serveur est à quinze pour cent. À 8 h 41, tout repart d'un coup. Le service informatique a ajouté de la mémoire, sans effet.

Expliquez ce qui se passe probablement, et pourquoi les journaux sont muets. Décrivez la démarche de diagnostic, en nommant les vues système que vous consultez et l'ordre dans lequel vous le faites. Dites ce que le fait que tout reparte d'un coup vous apprend. Proposez enfin deux corrections, l'une immédiate et l'autre durable.

M3. Coopérative Portneuf, le chargement nocturne qui n'aime pas être relancé. Le chargement nocturne lit une douzaine de fichiers plats, les insère dans autant de tables, puis reconstruit trois tables agrégées. Il met quarante minutes. Quand il échoue à mi-parcours, l'équipe le relance, et il arrive alors que certaines tables contiennent des lignes en double et que les agrégats ne correspondent plus au détail. La correction se fait à la main, et elle prend une demi-journée.

Concevez la reprise de ce chargement. Dites précisément où vous placez les transactions et pourquoi vous ne mettez pas tout dans une seule. Traitez le cas des doublons à l'insertion, celui des agrégats qui doivent rester cohérents avec le détail, et celui de la reprise après échec au milieu. Dites enfin comment l'équipe saura, au matin, si le chargement est allé au bout, sans avoir à comparer des comptes à la main. (avancé)

© 2026 iAlexMG. Tous droits réservés.

Corrigé

La numérotation est celle des énoncés.

Questions courtes

1. Un groupe de commandes qui réussit entièrement ou ne laisse aucune trace. Il n'y a pas de milieu.

Théorie : `Theorie_11_SQL.md`, section 1.

2. Il voit la table vide, ou partiellement remplie. Il n'échoue pas, il ne signale rien : il affiche zéro, ou un total incomplet, qui sont des réponses plausibles. Personne ne s'en aperçoit parce qu'aucune alerte ne se déclenche sur un chiffre qui a la bonne forme, et parce que le rapport suivant sera juste.

Théorie : `Theorie_11_SQL.md`, sections 1 et 9.

3. Qu'une transaction validée ne laisse jamais la base en violation d'une contrainte déclarée : clé primaire, clé étrangère, unicité, vérification. Elle ne protège de rien si vous n'avez rien déclaré, parce qu'elle n'invente aucune règle : c'est la seule des quatre lettres qui dépend entièrement de votre travail de modélisation.

Théorie : `Theorie_11_SQL.md`, section 2.

4. L'isolation. Les trois autres sont tout ou rien : une transaction est atomique ou ne l'est pas, durable ou ne l'est pas. L'isolation est un compromis entre la sécurité et le débit, et c'est pour cela qu'elle offre plusieurs niveaux.

Théorie : `Theorie_11_SQL.md`, sections 2 et 5.

5. À poser un point de reprise à l'intérieur d'une transaction, pour annuler une partie du travail sans tout perdre. Sur PostgreSQL, il devient indispensable dès qu'une commande échoue : la transaction entière refuse alors toute nouvelle commande, et seul un retour à un point de reprise permet de continuer sans tout annuler.

Théorie : `Theorie_11_SQL.md`, section 3.

6. Sur PostgreSQL. Il sait annuler un `CREATE TABLE` ou un `ALTER TABLE`, ce qui permet d'exécuter une migration de schéma entière dans une transaction : soit elle passe complètement, soit le schéma reste exactement dans son état de départ.

Théorie : `Theorie_11_SQL.md`, section 3.

7. Lecture sale, lecture non répétable, ligne fantôme, mise à jour perdue, anomalie d'écriture. La lecture sale ne se produit jamais sur PostgreSQL, même en demandant le niveau le plus permissif : la demande est acceptée et traitée comme une lecture validée.

Théorie : `Theorie_11_SQL.md`, section 4.

© 2026 iAlexMG. Tous droits réservés.

8. Les trois premières sont des anomalies de lecture : une transaction voit quelque chose qu'elle ne devrait pas voir. Les deux dernières sont faites de transactions individuellement correctes, dont seule la combinaison est fausse. Aucune ne fait quoi que ce soit d'illégal, ce qui rend le diagnostic beaucoup plus difficile.

Théorie : `Theorie_11_SQL.md`, section 4.

9. Deux médecins sont de garde et le règlement en exige au moins un. Chacun ouvre une transaction, compte les médecins de garde, en trouve deux, et se retire. Les deux valident, il ne reste personne, et aucune contrainte n'a été violée parce que la règle porte sur une propriété collective que le moteur ne connaît pas.

Théorie : `Theorie_11_SQL.md`, section 4.

10. La lecture validée. Elle empêche la lecture sale, et elle autorise encore la lecture non répétable et la ligne fantôme : chaque commande voit un instantané pris à son propre démarrage, donc deux commandes d'une même transaction peuvent voir deux états différents.

Théorie : `Theorie_11_SQL.md`, section 5.

11. Parce qu'elle prend un instantané complet à l'ouverture de la transaction, ce qui empêche aussi les lignes fantômes. La norme n'exige ce niveau que contre la lecture non répétable.

Théorie : `Theorie_11_SQL.md`, section 5.

12. Il ne bloque pas davantage : il laisse les transactions travailler et détecte, au moment de valider, si leur entrelacement ne correspond à aucun ordre séquentiel possible. Il annule alors l'une d'elles. L'obligation qui en découle est que l'application sache rejouer une transaction annulée, faute de quoi on a remplacé une donnée fausse par une erreur devant l'utilisateur.

Théorie : `Theorie_11_SQL.md`, section 5.

13. Un verrou partagé peut être tenu par plusieurs transactions à la fois, c'est le verrou de lecture ; un exclusif n'est tenu que par une seule, c'est celui de l'écriture. Un verrou de ligne ne bloque que la ligne visée, un verrou de table bloque tout le monde et apparaît notamment sur les commandes de structure.

Théorie : `Theorie_11_SQL.md`, section 6.

14. L'interblocage est une erreur, le blocage n'en est pas une. Un blocage se résout quand le verrou attendu se libère, et il peut durer indéfiniment sans que rien ne se plaigne. Un interblocage est une impasse dont aucune des deux transactions ne peut sortir ; le moteur la détecte, choisit une victime et l'annule, ce qui débloque l'autre.

Théorie : `Theorie_11_SQL.md`, section 6.

15. Verrouillez toujours les ressources dans le même ordre.

Théorie : `Theorie_11_SQL.md`, section 6.

© 2026 iAlexMG. Tous droits réservés.

16. Il n'écrit pas dans la ligne existante. Il écrit une nouvelle version de la ligne ailleurs, et marque l'ancienne comme périmée à partir de son numéro de transaction. Chaque version porte les numéros qui décident de sa visibilité pour chaque transaction.

Théorie : `Theorie_11_SQL.md`, section 7.

17. L'agréable : les lecteurs ne bloquent pas les écrivains, donc un rapport long ne gêne personne et n'attend personne. La désagréable : les versions périmées s'accumulent, la table grossit à nombre de lignes constant, et ses parcours ralentissent tant que le nettoyage n'est pas passé.

Théorie : `Theorie_11_SQL.md`, section 7.

18. Parce que l'index ne porte pas l'information de visibilité : le moteur qui trouve une entrée ne sait pas si la ligne correspondante est visible pour votre transaction, donc il ouvre la table pour vérifier. Il en est dispensé sur les pages marquées comme entièrement visibles, ce que fait le nettoyage. C'est donc un `VACUUM` qui fait baisser ce compteur.

Théorie : `Theorie_11_SQL.md`, section 7.

19. Parce que les écritures y arrivent par lots et que personne ne met à jour une ligne à la fois : deux transactions ne se disputent presque jamais la même ligne. Non, le voyage dans le temps ne remplace pas la transaction : il répare après coup, sur décision humaine, alors que la transaction empêche l'état intermédiaire d'exister. Un rapport lancé pendant cet état aura quand même donné un chiffre faux.

Théorie : `Theorie_11_SQL.md`, section 8.

20. Un chargement idempotent laisse la base dans le même état qu'il soit lancé une fois ou dix, avec la même entrée. Les trois motifs sont le remplacement complet de la table, la suppression puis la réinsertion de la tranche concernée, et la fusion par clé. Le `MERGE` exige que la clé soit unique dans la source, et ce contrôle appartient au chargement.

Théorie : `Theorie_11_SQL.md`, section 9.

Questions pratiques

P1. `[OLTP]`

```sql
-- Session A
BEGIN;
SELECT solde FROM labo11.compte WHERE id = 1;

-- Session B
UPDATE labo11.compte SET solde = solde + 10 WHERE id = 1;

-- Session A
SELECT solde FROM labo11.compte WHERE id = 1;    -- a changé
COMMIT;
```


© 2026 iAlexMG. Tous droits réservés.

Puis la même chose d'un cran plus haut :

```
-- Session A
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT solde FROM labo11.compte WHERE id = 1;

-- Session B
UPDATE labo11.compte SET solde = solde + 10 WHERE id = 1;

-- Session A
SELECT solde FROM labo11.compte WHERE id = 1;   -- inchangé
COMMIT;
SELECT solde FROM labo11.compte WHERE id = 1;   -- la nouvelle valeur apparaît
```

Théorie : `Theorie_11_SQL.md`, sections 4 et 5.

P2. `[OLTP]`

```
BEGIN;
UPDATE labo11.compte SET solde = solde - 100 WHERE id = 1;
UPDATE labo11.compte SET solde = solde + 100 WHERE id = 2;
COMMIT;
```

La contrainte `CHECK (solde >= 0)` fait le travail de refus : si le débit rend le solde négatif, la première commande échoue, la transaction passe en état d'erreur, et le `COMMIT` se comporte comme un `ROLLBACK`. Rien n'est écrit, et c'est exactement le comportement voulu.

Est également acceptable une réponse qui vérifie explicitement avant d'écrire, à condition de le faire sous verrou :

```
BEGIN;
SELECT solde FROM labo11.compte WHERE id = 1 FOR UPDATE;
-- l'application décide, puis :
UPDATE labo11.compte SET solde = solde - 100 WHERE id = 1;
UPDATE labo11.compte SET solde = solde + 100 WHERE id = 2;
COMMIT;
```

Sans le `FOR UPDATE`, la vérification ne vaut rien : une autre session peut vider le compte entre la lecture et l'écriture.

Théorie : `Theorie_11_SQL.md`, section 3.

P3. `[OLTP]` La production est celle de la partie 3 du laboratoire : deux sessions lisent la même valeur, chacune calcule son nouveau solde hors de la base, chacune écrit une valeur absolue, et la seconde écrase la première.

```
-- Correction 1 : le calcul dans le moteur.
UPDATE labo11.compte SET solde = solde - 30 WHERE id = 1;
```


© 2026 iAlexMG. Tous droits réservés.

```sql
-- Correction 2 : le verrou dès la lecture.
BEGIN;
SELECT solde FROM labo11.compte WHERE id = 1 FOR UPDATE;
UPDATE labo11.compte SET solde = solde - 30 WHERE id = 1;
COMMIT;
```

La première est la bonne chaque fois que le nouveau montant se déduit de l'ancien par une opération que le SQL sait exprimer : elle ne bloque presque rien et elle est atomique par construction. La seconde est la bonne quand le calcul se fait hors de la base, par exemple parce qu'une règle métier complexe doit être évaluée par du code applicatif avant de décider.

Une troisième réponse est recevable : le niveau sérialisable, qui détecte le conflit et annule l'une des deux transactions. Elle est correcte, et elle coûte une reprise à écrire.

Théorie : `Theorie_11_SQL.md`, sections 4 et 6.

P4. `[OLTP]`

```sql
SELECT pid,
       pg_blocking_pids(pid) AS bloque_par,
       state,
       now() - xact_start    AS duree_transaction,
       left(query, 80)       AS requete
FROM   pg_stat_activity
WHERE  cardinality(pg_blocking_pids(pid)) > 0
ORDER  BY duree_transaction DESC;
```

La colonne de durée n'était pas demandée, et elle est ce qui rend la requête utile en production : elle désigne la transaction ouverte depuis trop longtemps, qui est presque toujours la cause.

Théorie : `Theorie_11_SQL.md`, section 6.

P5. `[OLTP]` La production est celle de la partie 5 du laboratoire : A verrouille la ligne 1 puis demande la 2, B verrouille la 2 puis demande la 1.

Pour n'obtenir qu'un blocage, il suffit que les deux sessions prennent les verrous dans le même ordre :

```sql
-- Session A            -- Session B
BEGIN;                  BEGIN;
UPDATE ... WHERE id=1;  UPDATE ... WHERE id=1;   -- B attend ici
UPDATE ... WHERE id=2;
COMMIT;                 -- B repart, prend 1 puis 2
```

Ce qui a changé n'est pas le nombre de verrous ni leur nature, c'est uniquement l'ordre d'acquisition. B attend au lieu de s'engager dans une impasse.

Théorie : `Theorie_11_SQL.md`, section 6.

© 2026 iAlexMG. Tous droits réservés.

P6. [OLTP]

```sql
BEGIN;
DELETE FROM evenement WHERE jour = '2024-03-01';
INSERT INTO evenement (jour, source, montant)
SELECT '2024-03-01', source, montant FROM arrivee;
COMMIT;
```

Le BEGIN et le COMMIT font deux choses à la fois. Ils rendent le chargement atomique, donc un échec à l'insertion ne laisse pas la journée supprimée. Et ils ferment la fenêtre pendant laquelle un lecteur verrait la journée vide : sous MVCC, tout lecteur voit l'état d'avant jusqu'au COMMIT, puis l'état d'après.

L'idempotence, elle, était déjà là : la suppression de la tranche avant sa réinsertion garantit qu'un second passage produit le même résultat. Une bonne réponse le dit, et ajoute que le filtre du DELETE et celui des données insérées doivent porter sur exactement la même tranche, sans quoi le chargement laisse des lignes orphelines.

Théorie : Theorie_11_SQL.md, section 9.

P7. [OLTP] (avancé)

```sql
-- Le contrôle, avant tout le reste.
SELECT cle, COUNT(*) FROM source GROUP BY cle HAVING COUNT(*) > 1;

MERGE INTO cible AS c
USING source AS s ON c.cle = s.cle
WHEN MATCHED THEN UPDATE SET valeur = s.valeur, maj = now()
WHEN NOT MATCHED THEN INSERT (cle, valeur, maj) VALUES (s.cle, s.valeur, now());
```

Sans le contrôle, une source contenant deux fois la même clé fait échouer la commande sur PostgreSQL, avec un message indiquant qu'une ligne cible serait mise à jour deux fois. C'est la bonne nouvelle : le moteur refuse plutôt que de choisir au hasard. Sur d'autres moteurs, la même situation produit un résultat non déterministe, qui dépend de l'ordre de lecture et change d'une exécution à l'autre.

Une bonne réponse ajoute la parade quand les doublons sont attendus : on dédoublonne la source dans une CTE, en gardant la ligne la plus récente par clé, avec le ROW_NUMBER du Cours 04.

Théorie : Theorie_11_SQL.md, section 9.

P8. (avancé) Trois terrains, trois réponses.

Sur PostgreSQL, si la commande fautive était dans une transaction encore ouverte, un ROLLBACK suffit. Sinon, il n'y a rien dans le moteur : on restaure depuis une sauvegarde, ou on rejoue le journal des écritures si la configuration le permet. Autrement dit, il fallait y avoir pensé avant.

Sur l'entrepôt, il n'y a rien non plus, et pour une raison plus simple encore : le fichier a été modifié sur place. La seule parade est la copie du fichier avant l'opération risquée, ce qui prend une seconde sur 25 Mo. Là aussi, il fallait y avoir pensé avant.

© 2026 iAlexMG. Tous droits réservés.

Sur un grand entrepôt, et c'est la seule des trois réponses qui ne demande rien à préparer, le voyage dans le temps fait le travail. D'après la documentation de Snowflake, on constate l'état d'avant par `SELECT COUNT(*) FROM ma_table AT (OFFSET => -600)`, on le compare à l'état courant, puis on restaure par `CREATE OR REPLACE TABLE ma_table AS SELECT * FROM ma_table AT (OFFSET => -600)`. La condition est la durée de rétention : passé ce délai, les états anciens n'existent plus.

La leçon à retenir n'est pas le nom de la clause, c'est la question à se poser avant de lancer une commande destructrice sur un moteur qu'on ne connaît pas encore : qu'est-ce qui me rattrape ici, et est-ce que cela existe déjà ou est-ce que je dois le poser moi-même ?

La condition est la durée de rétention du voyage dans le temps sur cette table. Elle vaut un jour par défaut sur l'édition standard, et peut aller plus loin sur les éditions supérieures. Au delà, l'état n'existe plus et il faut une sauvegarde.

Est également acceptable une réponse qui emploie `BEFORE (STATEMENT => '<identifiant>')`, ce qui est plus précis qu'un décalage en secondes : on vise la commande fautive plutôt qu'un instant approximatif.

Théorie : `Theorie_11_SQL.md`, section 8.

Diagnostics

E1. Le `TRUNCATE` et l'`INSERT` sont deux commandes, donc deux transactions. Entre les deux, la table est vide, et tout rapport lancé à cet instant affiche zéro. Cela n'arrive qu'une fois par mois parce qu'il faut que quelqu'un consulte le tableau de bord pendant les quelques minutes que dure l'insertion, ce qui est rare à 2 h du matin mais pas impossible.

```
BEGIN;
TRUNCATE TABLE rapport_ventes;
INSERT INTO rapport_ventes SELECT ... FROM ...;
COMMIT;
```

Une bonne réponse note que `TRUNCATE` est annulable sur PostgreSQL, ce qui rend cette correction possible ; sur un moteur où il ne l'est pas, il faudrait remplacer le `TRUNCATE` par un `DELETE`, ou construire la table à côté et l'échanger par un renommage.

Une excellente réponse va plus loin et conteste la correction elle-même, parce qu'elle règle le symptôme et en crée un autre. `TRUNCATE` prend le verrou d'accès exclusif sur la table, mesuré : il bloque donc aussi les **lectures** jusqu'au `COMMIT`. Les lecteurs ne voient plus zéro, c'est vrai, mais ils ne voient plus rien du tout, et leur rapport attend la fin du chargement au lieu de répondre. On a remplacé un chiffre faux par un blocage, ce qui est préférable et reste un défaut.

Les deux écritures qui ne bloquent pas les lecteurs sous MVCC sont donc celles-ci. La première : `DELETE` puis `INSERT` dans une transaction, le `DELETE` ne prenant que des verrous de ligne, que les lectures ignorent. Elle coûte plus cher que le `TRUNCATE` et c'est le prix à payer. La seconde, et c'est celle d'un chargement de production : construire la nouvelle table à côté, sous un autre nom, puis l'échanger par un renommage dans une transaction courte. Les lecteurs voient l'ancienne table jusqu'au dernier instant, puis la nouvelle, et jamais rien entre les deux.

Théorie : `Theorie_11_SQL.md`, section 9.

© 2026 iAlexMG. Tous droits réservés.

E2. C'est la mise à jour perdue. Le scénario : deux sessions lisent 450, la première calcule 450 moins 100 et écrit 350, la seconde calcule 450 moins 50 et écrit 400. Les deux retraits ont eu lieu, un seul se voit, et le solde final est supérieur à ce qu'il devrait être.

Deux corrections. Laisser le moteur calculer, avec `UPDATE compte SET solde = solde - 100 WHERE id = 1`, ce qui rend l'opération atomique. Ou verrouiller dès la lecture, avec `SELECT solde FROM compte WHERE id = 1 FOR UPDATE`, ce qui oblige la seconde session à attendre et à relire la valeur à jour.

Théorie : `Theorie_11_SQL.md`, sections 4 et 6.

E3. Le niveau sérialisable ne se contente pas de protéger les lectures : il surveille les dépendances entre transactions et annule celles dont l'entrelacement n'est pas sérialisable, y compris quand la transaction annulée ne fait que lire. Sur une base active, un rapport long croise beaucoup d'écritures, donc il devient une victime probable.

La correction la plus économique est le niveau juste en dessous, la lecture répétable, qui donne exactement ce que le développeur cherchait : un instantané cohérent pour toute la durée du rapport, sans risque d'annulation.

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT ... ;
SELECT ... ;
COMMIT;
```

Une bonne réponse ajoute que si l'on tient au niveau sérialisable, PostgreSQL permet de déclarer la transaction en lecture seule et différée, ce qui la met à l'abri des annulations au prix d'une attente au démarrage.

Théorie : `Theorie_11_SQL.md`, section 5.

E4. L'`ALTER TABLE` prend un verrou exclusif sur la table dès son exécution, et une transaction garde ses verrous jusqu'au `COMMIT`. Le verrou est donc tenu pendant les vingt-cinq minutes de l'`UPDATE`, et toute commande qui touche à `commande`, y compris une simple lecture applicative, attend derrière.

Les journaux sont muets parce qu'attendre un verrou n'est pas une erreur. Rien n'échoue, rien n'expire si aucun délai n'est configuré, et l'application semble simplement figée.

Ce qu'il fallait faire : séparer les deux opérations. L'ajout de colonne avec une valeur par défaut est presque instantané sur les versions récentes de PostgreSQL et peut se faire seul. La mise à jour des 40 millions de lignes se fait ensuite par lots, chaque lot dans sa propre transaction courte, ce qui laisse respirer les autres sessions entre deux. Et l'ensemble se lance hors des heures ouvrables.

Théorie : `Theorie_11_SQL.md`, section 6.

E5. La cause la plus probable est le gonflement par versions périmées. La table est mise à jour en continu, donc chaque modification laisse une version morte, et si le nettoyage n'arrive pas à suivre, la table occupe beaucoup plus d'espace que son contenu ne le justifie. Les parcours ralentissent en proportion de l'espace, pas du nombre de lignes, ce qui explique exactement l'écart décrit.

```
SELECT relname, n_live_tup, n_dead_tup,
       last_autovacuum, last_vacuum,
```


© 2026 iAlexMG. Tous droits réservés.

```
       pg_size_pretty(pg_relation_size(relid)) AS taille
FROM   pg_stat_user_tables
WHERE  relname = 'audit';
```

Deux traitements. Un `VACUUM` ordinaire, éventuellement plus fréquent en ajustant les paramètres du démon pour cette table : il ne rend pas l'espace au système mais le rend réutilisable, et il ne bloque personne. Ou un `VACUUM FULL`, qui réécrit la table et la rend compacte, au prix d'un verrou exclusif pendant toute la durée de l'opération, ce qui est inenvisageable en journée sur 900 millions de lignes.

Une bonne réponse choisit le premier et traite la cause : c'est le réglage du nettoyage qui est insuffisant, pas la table qui est malade.

Théorie : `Theorie_11_SQL.md`, section 7.

E6. Sur nos deux terrains, tout est annulé, et le développeur a raison de s'y attendre : PostgreSQL comme l'entrepôt savent annuler une commande de structure, ce que le Cours 03, section 8, a mesuré des deux côtés. La table `cible` retrouve son contenu d'avant, et les lignes de `journal` reviennent.

Sur un moteur qui valide le DDL d'office, le `DELETE` est annulé et le `CREATE OR REPLACE TABLE` ne l'est pas. La commande de structure valide implicitement la transaction en cours au moment où elle s'exécute : `cible` est remplacée définitivement, et le `ROLLBACK` n'annule que ce qui suivait cette validation implicite. C'est le cas d'Oracle, de MySQL et de Snowflake d'après leur documentation, où seul le voyage dans le temps permettrait ensuite de récupérer l'état précédent de `cible`.

La règle d'écriture qui rend ce bloc sûr partout tient en une phrase : ne mélangez jamais du DDL et du DML dans la même transaction. Faites la structure d'abord, dans son propre bloc, vérifiez, puis faites les données dans une transaction séparée. Vous perdez l'atomicité de l'ensemble, que tous les moteurs ne vous offraient de toute façon pas, et vous gagnez un comportement identique partout. Le confort dont vous disposez chez vous est ici le vrai danger : il produit du code qui marche sur votre machine et qui détruit des données ailleurs.

Théorie : `Theorie_11_SQL.md`, section 8.

Questions à développer

A1. Le réglage échange de la sécurité contre du débit, et il le fait toujours dans le même sens : plus le niveau est strict, moins il se produit d'anomalies, et plus il se produit d'attentes ou d'annulations. Un niveau permissif laisse passer des lectures incohérentes mais ne fait attendre personne ; un niveau strict garantit la cohérence en payant, soit par des verrous tenus plus longtemps, soit par des transactions annulées à la validation. Il n'y a pas de bon niveau dans l'absolu, il y a un niveau adapté à ce qu'on fait.

Pour une application de caisse, la lecture validée suffit dans la plupart des cas, à condition que les écritures soient formulées de façon atomique, avec le calcul dans le moteur plutôt que dans le code. On monte au verrouillage explicite sur les rares opérations où une règle doit être vérifiée avant d'écrire.

Pour un rapport analytique multi-requêtes, la lecture répétable est le bon choix, et c'est le cas le plus mal servi en pratique. Elle donne un instantané cohérent pour toute la durée du rapport, elle ne bloque personne sous MVCC, et elle coûte deux mots dans le script.

© 2026 iAlexMG. Tous droits réservés.

Pour un chargement nocturne, la lecture validée suffit, parce que le chargement est seul ; ce qui compte n'est pas le niveau mais la portée de la transaction, choisie pour qu'aucun lecteur ne voie jamais un état intermédiaire.

Théorie : `Theorie_11_SQL.md`, sections 2 et 5.

A2. Un chargement finit toujours par être relancé, et trois raisons suffisent à s'en convaincre. Il échoue à mi-parcours, pour une raison qui n'a rien à voir avec lui, un disque plein ou un réseau qui tombe, et il faut bien le reprendre. Un fichier source arrive deux fois, parce que le fournisseur a renvoyé un lot ou parce qu'une reprise amont l'a réémis. Et quelqu'un le relance par prudence, parce qu'un doute existe sur son exécution de la veille, ce qui est le cas le plus fréquent et le moins prévu.

Un chargement non idempotent transforme chacune de ces trois situations ordinaires en incident de données, avec une correction manuelle et une perte de confiance dans les chiffres. L'idempotence n'est donc pas une élégance, c'est ce qui rend le chargement exploitable par quelqu'un d'autre que son auteur.

Sur une table de faits de 200 millions de lignes alimentée par jour, le remplacement complet est hors de question : reconstruire 200 millions de lignes chaque nuit pour en ajouter quelques centaines de milliers est un gaspillage. La suppression et la réinsertion de la tranche du jour est la bonne réponse dans la majorité des cas : elle est simple, elle se raisonne facilement, et son coût est proportionnel à la tranche. La fusion par clé devient préférable quand la source livre des corrections sur des jours antérieurs, parce que la tranche à recharger n'est alors plus connue à l'avance.

Une réponse qui choisit la fusion est également recevable si elle traite le contrôle d'unicité de la source et le coût de la comparaison sur 200 millions de lignes.

Théorie : `Theorie_11_SQL.md`, section 9.

A3. Une transaction empêche : elle fait en sorte qu'un état intermédiaire n'existe jamais pour personne. Un lecteur voit l'état d'avant, puis l'état d'après, et rien entre les deux. Elle ne répare rien, parce qu'elle n'a rien à réparer.

Le voyage dans le temps répare : il conserve les états passés et permet d'y revenir après avoir constaté une erreur. Il n'empêche rien, parce que l'état fautif a bel et bien existé, et parce que tout ce qui l'a lu pendant qu'il existait a produit un résultat faux.

L'une ne remplace donc pas l'autre parce qu'elles n'agissent pas au même moment. Le voyage dans le temps est excellent contre la faute humaine, la commande lancée sur la mauvaise table, le fichier chargé deux fois, et il est sans effet contre le rapport qui a lu pendant la fenêtre.

Quand on ne dispose que de la seconde, ce qu'un ingénieur analytique doit mettre en place est ce qui supprime la fenêtre : construire la nouvelle version de la table à côté, puis la substituer par une opération unique, un échange de nom ou une écriture atomique. Le lecteur voit alors l'ancienne version ou la nouvelle, jamais un entre-deux. Une bonne réponse mentionne aussi la publication par vue, qui permet de basculer la définition d'une vue d'une table vers l'autre en une commande.

Théorie : `Theorie_11_SQL.md`, sections 1 et 8.

A4. Trois raisons expliquent sa rareté. La première est le coût en développement : le niveau sérialisable oblige l'application à savoir rejouer une transaction annulée, ce qui suppose que le travail à rejouer soit connu et rejouable, et beaucoup d'applications ne sont pas écrites ainsi. La deuxième est le taux d'annulation, qui grimpe vite quand les transactions sont longues ou qu'elles lisent beaucoup, ce qui est exactement le cas des rapports. La troisième est la

© 2026 iAlexMG. Tous droits réservés.

méconnaissance : les anomalies qu'il empêche, la mise à jour perdue et l'anomalie d'écriture, sont invisibles, et personne ne demande une protection contre un problème qu'il n'a jamais vu.

On le recommande malgré tout dans un cas précis : quand une règle métier porte sur un ensemble de lignes plutôt que sur une ligne, qu'aucune contrainte de la base ne peut l'exprimer, et que sa violation coûte cher. La règle du médecin de garde en est le modèle, et on la retrouve dans toute réservation à capacité limitée, tout budget partagé, toute affectation exclusive.

Avant de l'activer, il faut avoir écrit trois choses. La détection de l'erreur de sérialisation par son code, et non par un message générique. La reprise, avec un nombre d'essais borné et une attente croissante entre deux essais. Et la garantie que la transaction ne produit aucun effet hors de la base avant sa validation : une transaction qui envoie un courriel au milieu ne peut pas être rejouée, et c'est l'erreur de conception qui rend le reste inutile.

Théorie : `Theorie_11_SQL.md`, section 5.

Mises en situation

M1. Réponse défendable.

L'anomalie est l'anomalie d'écriture, la même que celle des médecins de garde. Le scénario : deux sessions ouvrent chacune une transaction, comptent les billets vendus, trouvent le même nombre, concluent chacune qu'il reste de la place, et insèrent chacune leur réservation. Les deux transactions écrivent des lignes différentes, donc aucun verrou n'entre en conflit, donc rien ne les empêche de valider. Le samedi soir, à l'ouverture des ventes, la fréquence des réservations rend l'entrelacement quasi certain ; le reste de la semaine, deux réservations tombent rarement dans la même fraction de seconde, et le problème dort.

Trois corrections de natures différentes.

Verrouiller la ligne du spectacle avant de compter, avec un `SELECT … FOR UPDATE` sur la ligne du spectacle dans la table des spectacles. Toutes les réservations d'un même spectacle se sérialisent, et celles des autres spectacles ne sont pas gênées. Le coût est une attente par réservation, de l'ordre de quelques millisecondes.

Passer la transaction au niveau sérialisable. Le moteur détecte le conflit et annule l'une des deux, et l'application rejoue. Le coût est un taux d'annulation qui monte avec l'affluence, donc précisément au pire moment, et il faut avoir écrit la reprise.

Déplacer la règle dans la base, en tenant un compteur de places restantes sur la ligne du spectacle et en le décrémentant par un `UPDATE` atomique assorti d'une contrainte de positivité. Le compteur devient la ressource disputée, l'écriture est atomique, et la contrainte refuse la vente de trop.

La recommandation raisonnable est la troisième, avec la première en repli. Elle transforme une règle collective invisible en une contrainte que le moteur sait faire respecter, ce qui est toujours préférable, et son coût est le même quelle que soit l'affluence.

Ce qu'il faut mesurer est la partie que l'énoncé rend difficile. Comme l'incident était hebdomadaire, attendre qu'il ne se reproduise pas ne prouve rien avant plusieurs mois. Il faut donc un contrôle permanent qui compare, pour chaque spectacle, le nombre de réservations à la jauge, et qui alerte à la première

© 2026 iAlexMG. Tous droits réservés.

violation. Une bonne réponse ajoute qu'on peut reproduire le problème avant la correction, par un test de charge qui lance des réservations concurrentes, ce qui donne une preuve immédiate au lieu d'une absence de preuve étalée sur un trimestre.

Théorie : `Theorie_11_SQL.md`, sections 4 et 5.

M2. Réponse défendable.

Ce qui se passe est très probablement une longue transaction qui tient un verrou, et derrière laquelle tout le monde s'accumule. Le créneau de 8 h 10 à 8 h 40 suggère un traitement programmé, un import du matin ou une opération de maintenance, qui ouvre une transaction et la garde ouverte pendant sa durée.

Les journaux sont muets parce qu'attendre un verrou n'est pas une erreur. Aucune commande n'échoue, aucune ne dépasse un délai puisque aucun délai n'est configuré, et l'application attend simplement. Le processeur à quinze pour cent confirme le diagnostic : personne ne travaille, tout le monde attend. C'est aussi pourquoi ajouter de la mémoire n'a rien changé.

La démarche de diagnostic, dans l'ordre. Pendant l'incident, on regarde `pg_stat_activity` avec `pg_blocking_pids` pour obtenir la chaîne d'attente et remonter à la session en tête. On regarde ensuite depuis quand cette session a ouvert sa transaction, avec `xact_start`, et quelle est sa dernière commande. On regarde enfin `pg_locks` pour savoir quel objet est verrouillé et sous quel mode, ce qui identifie la table et donc le traitement en cause.

Le fait que tout reparte d'un coup est l'indice le plus parlant, et il faut le dire explicitement : c'est la signature d'un verrou relâché, donc d'une transaction qui se termine, et non d'une saturation de ressource, qui se résorberait progressivement.

Deux corrections. Immédiate : décaler le traitement hors des heures d'ouverture, ce qui ne règle rien mais rend le problème invisible aux utilisateurs et laisse le temps de faire mieux. Durable : découper le traitement en transactions courtes, une par lot, de façon que le verrou soit relâché régulièrement, et vérifier qu'il ne prend pas de verrou de table là où un verrou de ligne suffirait.

Théorie : `Theorie_11_SQL.md`, section 6.

M3. Réponse défendable.

Le découpage des transactions est le cœur de la réponse, et l'erreur serait de tout mettre dans une seule. Une transaction de quarante minutes tient ses verrous quarante minutes, empêche le nettoyage de récupérer quoi que ce soit pendant ce temps, et fait tout perdre sur un échec à la trente-neuvième minute. Le bon découpage est d'une transaction par table chargée, plus une transaction par table agrégée, chacune atomique et chacune rejouable seule.

Les doublons se traitent à la source du problème, qui est la réinsertion sans suppression. Chaque table se charge par suppression de la tranche puis réinsertion, dans la même transaction, ou par fusion sur la clé métier quand la tranche n'est pas identifiable. Dans les deux cas, une contrainte d'unicité sur la clé métier doit exister dans la base : c'est elle qui transforme un doublon en erreur bruyante plutôt qu'en ligne dupliquée silencieuse.

Les agrégats se traitent en les reconstruisant à partir du détail plutôt qu'en les incrémentant, et en les reconstruisant après que le détail est validé. Reconstruire coûte plus cher qu'incrémenter, et c'est le prix de la cohérence : un agrégat incrémental relancé deux fois compte deux fois. Chaque reconstruction se fait dans sa propre transaction, de façon qu'aucun lecteur ne voie un agrégat vide.

© 2026 iAlexMG. Tous droits réservés.

La reprise après échec au milieu découle du découpage : puisque chaque étape est atomique et idempotente, la reprise consiste à relancer le chargement complet. C'est la solution la plus simple et la plus sûre, et elle est possible précisément parce que chaque étape peut être rejouée sans dommage. Une variante plus fine, acceptable, consiste à tenir une table de suivi des étapes accomplies pour ne rejouer que ce qui manque, à condition que cette table soit mise à jour dans la même transaction que l'étape qu'elle décrit.

Enfin, ce qui dit au matin que le chargement est allé au bout : une table de journal, alimentée par le chargement lui-même, qui enregistre pour chaque étape son heure de début, son heure de fin, le nombre de lignes traitées et son statut, et une dernière ligne qui marque l'achèvement global. L'équipe regarde cette ligne, pas des comptes. Une bonne réponse ajoute des contrôles de cohérence automatiques après le chargement, par exemple la comparaison entre la somme du détail et celle de l'agrégat, dont le résultat est écrit dans le même journal.

Théorie : `Theorie_11_SQL.md`, section 9.

© 2026 iAlexMG. Tous droits réservés.